

Multi-Region Customer Placement Design

CR/CRG-Weighted Region Selection for Non-Paired Azure Regions

Classification: Principal Cloud Architect

Status: Architecture Design — Ready for Engineering Review

Depends On: `azure_cr_management_engine_design.md` , `agent.md` (CR Sharing Knowledge Base), POC test framework `cr_poc_tests/`

Contents

- Executive Summary
 - Requirements
 - WAF Assessment
 - Quota Group Architecture (Two-Group Model)
 - Core Constraints Model
 - Regional State Model
 - The Placement Scoring Formula
 - Selection Algorithm
 - Worked Examples — 3 and 4 Regions
 - Rebalancing and Drift Handling
 - Integration with ACRME Engine
 - Implementation Guidance
 - Decision Log
-

Design Update — August 2026 (Pass 1): Incorporated the **Two-Group Quota Architecture** (D6/D7). The per-region quota model now uses two `Microsoft.Quota/groupQuotas` groups (Prod | NonProd+DR). HC-3, `Quota_Score` , and `RegionalSnapshot` revised accordingly. Legacy per-subscription fields retained as `[Legacy]` .

Design Update — August 2026 (Pass 2): Incorporated the **CR/CRG Architecture** decisions (D8/D9). HC-1 constraint `DR_region ≠ NonProd_region` removed; HC-6 (`DR_COVERAGE_FLOOR`) added. Generic `PS(r,E)` formula replaced by env-type-specific `PS_Prod` / `PS_NonProd` / `PS_DR` formulas. `RegionalSnapshot` extended with per-CRG-type sub-schema (16 new fields). DR ratio parameters `dr_ratio_min=0.30` / `dr_ratio_max=0.40` / `dr_ratio_target=[0.30,0.40]` formally defined. Changes marked with `[UPDATED]` , `[NEW]` , and `[NEW – D8/D9]` tags.

Executive Summary

The scenario involves **3 or 4 non-paired Azure regions** acting simultaneously as hosts for Production, Non-Production, and DR workloads — but for **different customers**. Every customer's three environments (Prod, NonProd, DR) must land in **three distinct regions**. The engine must — given a customer's chosen Prod region — automatically and optimally select the NonProd and DR regions using a **CR/CRG capacity-weighted scoring function**, distributing load across the estate over time.

The solution presented here defines:

1. A **hard constraint model** (the “no same-region” rule + zone isolation).
2. A **Regional Capacity Weight (RCW)** computed per region in real time from live CRG/CR state.
3. A **Placement Score Formula** with five configurable weighted sub-scores.
4. A **sequential selection algorithm** (Prod → NonProd → DR) that consumes the score.
5. A **distribution balancer** to prevent region starvation and score convergence.

The design extends the existing **EPIC-07 Placement Engine** (stories E07-S01 through E07-S09) and the **EPIC-08 DR Orchestrator** (E08-S01 through E08-S09) already specified in the management engine design.

Requirements

Functional

- **R1:** Customer selects Prod region from the available region set.
- **R2:** Engine selects NonProd and DR regions automatically, never the same as Prod. NonProd and DR may share a region to enable DR overflow capacity reuse from the NonProd CRG. [Updated — see D8]
- **R3a:** With 2 regions (current model for all geographies except US): Prod occupies one region; Non-Prod/CVAL and DR co-locate in the other — deterministic and mandatory (no choice), per PLC-010a. HC-6/HC-7 verify the co-located pool can absorb DR demand.
- **R3:** With 3 regions (current model for US): all three regions are used, one per environment, per customer (no choice).
- **R4:** With 4 regions: engine selects the optimal 2 from the remaining 3 regions using the weighted formula.
- **R5:** Selection is weighted against live CR/CRG capacity state (headroom, utilization, sharing headroom).
- **R6:** Selection must also account for quota headroom, zone diversity, and DR buffer compliance.
- **R7:** The formula must be configurable (weights, thresholds) via `PlacementPolicy` (FR-6).
- **R8:** Engine must detect and handle region exhaustion (all candidate regions saturated).

Non-Functional

- **NFR-R1:** Selection must complete in < 500 ms (consistent with NFR-2 in the engine design).
 - **NFR-R2:** Regional weights must reflect state no older than the last reconciliation cycle (5 min).
 - **NFR-R3:** The algorithm must be deterministic given the same regional state snapshot — auditable.
-

WAF Assessment [UPDATED]

Security

- Region selection is an engine-level decision; the customer only sees the assignment, not the scoring data of other customers. Customer isolation is maintained: no customer data leaks through score output.
- RBAC: the placement API enforces `Operator` or `Consumer` role; region weight data is `Reader` -only.
- Audit: every placement decision is written to `OperationRecord` with `before_state` (scores) and `after_state` (assignment).

Reliability

- Non-paired regions remove the guaranteed geographic failover that Azure Paired regions provide. The DR region selection formula must therefore prioritize **maximum geographic distance** (proximity penalty) and **independent failure domains** as first-order constraints, not secondary ranking criteria.
- The `DRCapacityPair` model already enforces `dr_cr.quantity >= primary.quantity × dr_buffer_pct`; this formula feeds into it.
- 3-region model: a dual-region outage (2 of 3 regions fail) leaves a customer with no functional environment. This is a known limitation of 3-region non-paired estates — must be documented in the SLA.

Performance Efficiency

- Regional state snapshot is cached in Redis (refreshed every reconciliation cycle, 5 min). Placement scoring reads from cache, not live ARM; latency is $O(R \times C)$ where R = region count, C = CRG count per region.
- With 3–4 regions and a practical CRG count per region (<100), scoring is sub-millisecond on cached data.

Cost Optimization

- The scoring formula includes a Cost sub-score that directs placements toward lower-cost regions within acceptable capacity bounds.
- Non-prod placements can be directed toward regions with lower on-demand pricing, while DR placements are directed toward the lowest-cost region that still meets the buffer threshold.
- Chargeback attribution (EPIC-10) captures cost per region per customer; feeds into forecast and right-sizing.

Operational Excellence

- The formula is expressed as a `PlacementPolicy` document (config-as-code), auditable and version-controlled.
 - Every selection is written to the operation log with full score breakdown, enabling post-hoc analysis of distribution patterns and policy tuning.
 - The Distribution Score sub-component is a built-in fairness mechanism, automatically correcting imbalances without operator intervention.
-

Quota Group Architecture (Two-Group Model)

[Documented – GA with regional restrictions] — Validate Quota Group GA availability in target regions via POC-30 before committing this as a production dependency.

Foundational Principle

Each region has exactly **two Azure Quota Groups** (`Microsoft.Quota/groupQuotas`), one per environment class. This maps directly to the three-CRG model:

Prod CRG	↔	Prod Quota Group	(1:1 isolated)
NonProd CRG	↔	NonProd+DR Group	(shared pool)
DR CRG	↔	NonProd+DR Group	(shared pool)

The group boundary enforces Prod isolation at the Azure control plane level — not just in engine logic. NonProd and DR share a pool because the engine owns internal allocation decisions between them; Azure enforces only the combined ceiling.

Group Definitions Per Region

```
GROUP 1 – Prod Quota Group
Members:    Provider subscription (Prod CRG owner)
Budget:     Prod_CRG_quantity × vCPU_per_instance × (1 + prod_growth_buffer)
Backs:      Prod CRG reservations only
Isolation:  Hard ceiling – NonProd and DR can never consume from this group

GROUP 2 – NonProd+DR Quota Group
Members:    NonProd subscription + DR subscription
Budget:     (NonProd_CRG_qty × vCPU) + (DR_CRG_qty × vCPU) + emergency_transfer_headroom_vcpu
Backs:      NonProd CRG reservations + DR CRG reservations
Shared:     Both NonProd and DR draw from the same group budget
```

DR Floor — Engine-Enforced Sub-Limit Within the NonProd+DR Group

[Important – Derived] Azure Quota Groups do not natively support intra-group sub-reservations. The engine is the sole enforcer of the DR floor within the shared group.

```
DR_FLOOR_VCPU(region) =
    potential_dr_demand(region) × vCPU_per_instance × dr_ratio_max
```

Where:

```
potential_dr_demand = Σ prod_allocated for all customers where dr_region =
this_region
dr_ratio_max        = 0.40 (upper bound; use max to never undersize the floor)
vCPU_per_instance   = SKU-specific vCPU count
```

```
Effective_NonProd_Quota_Ceiling = NonProd_DR_Group_limit - DR_FLOOR_VCPU
```

ENFORCEMENT:

NonProd CRG expansion is BLOCKED when:

```
nonprod_quota_used + (new_qty × vCPU_per_instance) > Effect-
ive_NonProd_Quota_Ceiling
```

Alert: DRFloorViolationDetected (Severity: Critical)

Trigger: nonprod_quota_used > effective_nonprod_ceiling

Action: Engine blocks further NonProd CRG quantity increases

Why This Model Makes Tier 3 Emergency Capacity Transfer Truly Quota-Neutral

WITHOUT Quota Groups (old model):

Reduce NonProd CR qty 100→0: releases quota to NonProd subscription ledger

Increase DR CR qty 30→80: consumes quota from DR subscription ledger

→ Two separate Azure quota accounts must coordinate

→ Risk: DR sub quota increase request may be throttled or require approval (hours of RTO)

WITH NonProd+DR Quota Group:

Reduce NonProd CR qty 100→0: releases vCPU back to GROUP budget (same pool)

Increase DR CR qty 30→80: consumes from the SAME GROUP budget

→ Net change to group: headroom improves

→ No Azure quota increase request needed

→ ARM operations are the only latency – minutes, not hours

→ Tier 3 Emergency Transfer RTO gated only by ARM, not quota approval

QuotaGroup Entity (Cosmos DB)

```
QuotaGroup {
  group_id:          GUID (PK)
  region:            String
  group_type:        Enum [Prod, NonProdDR]
  member_subscription_ids: GUID[]
  arm_group_resource_id: String    ← /providers/Microsoft.Quota/groupQuotas/{id}
  group_limit_vcpu:   Int
  group_used_vcpu:    Int
  group_headroom_vcpu: Int        ← derived: limit - used

  // NonProdDR group fields only
  nonprod_quota_used_vcpu: Int
  dr_quota_used_vcpu:      Int
  dr_floor_vcpu:           Int
  effective_nonprod_ceiling: Int
  nonprod_headroom_vcpu:   Int
  dr_headroom_vcpu:        Int
  dr_floor_compliant:      Boolean

  last_synced_at:         DateTime
}
```

Quota Group Sizing Formulas

```
Prod_Group_Limit(region) =
  Prod_CRG_quantity(region) × vCPU_per_instance × (1 + prod_growth_buffer)
  prod_growth_buffer: 0.20 (20% headroom above current provisioned)

NonProd_DR_Group_Limit(region) =
  NonProd_CRG_quantity(region) × vCPU_per_instance × (1 + nonprod_growth_buffer)
+ DR_CRG_quantity(region) × vCPU_per_instance
+ emergency_transfer_headroom_vcpu

emergency_transfer_headroom_vcpu:
  max_emergency_transfer_qty × vCPU_per_instance
  (quota budget for Tier 3 DR CRG expansion beyond pre-positioned quantity)
  Without this term, Tier 3 has no quota room to expand the DR CRG during an emer-
  gency.
```

Validation Required

- **POC-30:** Quota Group GA availability — confirm `Microsoft.Quota/groupQuotas` available in all target regions
 - **POC-31:** NonProdDR group decomposition — confirm per-subscription usage sums to group total; validate quota releases back to group pool after CR qty reduction
 - **POC-32:** DR floor enforcement timing — measure quota propagation latency after CR qty→0; bounds Tier 3 RTO
 - **POC-33:** Quota increase targeting — confirm `POST Microsoft.Quota/groupQuotas/{id}/quota` is correct group-level increase endpoint
-

Core Constraints Model [UPDATED]

Before scoring begins, hard constraints eliminate ineligible candidates entirely. Soft objectives then rank the survivors.

Hard Constraints (eliminate — no override)

HC-1 REGION_SEPARATION [UPDATED – see D8]:
 NonProd_region $\neq$ Prod_region
 DR_region $\neq$ Prod_region
 [Constraint DR_region $\neq$ NonProd_region REMOVED – NonProd and DR may share a region]
 → NonProd/DR co-location is permitted to allow DR overflow capacity reuse from the NonProd CRG.
 → With 2 regions (current model for all geographies except US): Prod is isolated in one region;
 NonProd/CVAL and DR co-locate in the other, deterministically and mandatorily (PLC-010a).
 HC-6/HC-7 verify the shared co-located pool can absorb the customer's DR demand.
 → With 3 regions (current model for US): Prod is isolated; NonProd and DR both draw from the remaining 2 regions
 (they may land on the same region or on different ones – determined by HC-6 and PS score).
 → With 4 regions: Prod eliminates 1; NonProd selects from 3; DR may share with NonProd
 or use the remaining regions – whichever satisfies HC-6 and maximises PS_DR.

HC-2 CAPACITY_FLOOR:
 CR headroom in candidate region $\geq$ minimum_reservation_units
 (configurable; default = $2 \times \text{requested_vm_count} \times \text{SKU_vCPU_count}$)
 → Regions at capacity floor are excluded from scoring entirely.

HC-3 QUOTA_FLOOR [UPDATED – reads from Quota Groups, not per-subscription QuotaRecord]:
 For Prod placement in region R:
 REJECT if: $\text{Prod_Group_headroom}(R) < \text{requested_vm_count} \times \text{vCPU_per_instance}$
 For NonProd placement in region R:
 REJECT if: $\text{nonprod_headroom}(R) < \text{requested_vm_count} \times \text{vCPU_per_instance}$
 (nonprod_headroom uses effective_nonprod_ceiling – floor-adjusted, not raw group_limit)
 For DR placement in region R:
 REJECT if: $\text{dr_headroom}(R) < \text{target_dr_qty} \times \text{vCPU_per_instance}$
 (target_dr_qty = $\text{customer_prod_vm_count} \times \text{dr_ratio}$)
 Source: QuotaGroup entity (Cosmos DB), synced to RegionalSnapshot (Redis).
 Legacy note: Prior field QuotaRecord.deployment_headroom is superseded for placement checks.
 It is retained for per-subscription quota monitoring only.

HC-4 DR_SEPARATION_CLASS (non-paired regions only):
 DR_region must have Separation_Class(DR_region, Prod_region) = HIGH
 → Defined below; ensures non-correlated failure domains.

HC-5 ZONE_AVAILABILITY:
 Candidate region must have ≥ 2 physical Availability Zones
 → Single-zone regions are excluded for all non-dev environments.

HC-6 DR_COVERAGE_FLOOR [NEW – see D8]:
 REJECT DR placement in region R if combined DR + NonProd overflow capacity is insufficient to absorb the new customer's DR demand:
 $\text{dr_crg_free_slots}(R) + \text{nonprod_crg_effective_free}(R) < \text{customer_requested_dr_slots}$
 where:
 $\text{customer_requested_dr_slots} = \text{prod_vm_count} \times \text{dr_ratio_max}$ (target = upper bound, 0.40)
 $\text{nonprod_crg_effective_free} = \text{nonprod_crg_free_slots} - \text{non-}$

`prod_crg_dr_overflow_reserve`

Rationale: Since DR and NonProd may share a region (HC-1 constraint removed), the engine

must verify that the combined capacity pool can absorb the customer's DR need –

either from the DR CRG directly, or from the NonProd CRG's reserved overflow headroom.

This constraint operationalises HC-1 co-location at the capacity selection level.

Requires: per-CRG-type snapshot fields (`nonprod_crg_free_slots`, `nonprod_crg_effective_free`,

`nonprod_crg_dr_overflow_reserve`, `dr_crg_free_slots` – see Regional State Model).

HC-7 DR_FLOOR_INTEGRITY [NEW]:

REJECT NonProd placement in region R if placing the requested quantity

would push `nonprod_quota_used` above `effective_nonprod_ceiling`:

$\text{nonprod_quota_used}(R) + (\text{requested_vm_count} \times \text{vCPU_per_instance}) > \text{effective_nonprod_ceiling}(R)$

Rationale: Prevents NonProd allocation from encroaching on the protected DR quota floor.

This constraint is evaluated AFTER HC-3 – HC-3 checks raw headroom; HC-7 checks floor compliance.

Soft Objectives (rank survivors via score)

The five sub-scores ranked by the Placement Score Formula (PSF).

Regional State Model [UPDATED]

The engine maintains a **Regional State Snapshot** for each managed region, refreshed every reconciliation cycle. This snapshot is what the formula reads from — it is a denormalized Redis cache of the Cosmos DB / PostgreSQL entity state.

The snapshot has been extended with **Quota Group fields** (populated by the extended Quota Sync Worker, E03-S10). Legacy per-subscription quota fields are retained for monitoring and audit but are **superseded for all HC-3 and Quota_Score calculations** by the group-level fields.

```

RegionalSnapshot {
    region:                String
    total_crg_count:        Int          ← count of all CRGs in region
    total_cr_count:         Int
    total_quantity:         Int          ←  $\sum cr.actual\_quantity$  across all CRs
    total_allocated:        Int          ←  $\sum cr.allocated\_count$  across all CRs
    total_free_slots:       Int          ←  $total\_quantity - total\_allocated$ 

    // — QUOTA GROUP FIELDS [NEW] — primary source for HC-3, HC-7, Quota_Score —
    // Prod Quota Group
    prod_group_limit_vcpu:   Int          ← Microsoft.Quota/groupQuotas — Prod group
limit
    prod_group_used_vcpu:    Int          ← Prod CRG quantity × vCPU (all Prod CRs in
region)
    prod_group_headroom_vcpu: Int          ←  $prod\_group\_limit - prod\_group\_used$ 

    // NonProd+DR Quota Group
    nonprod_dr_group_limit_vcpu: Int      ← NonProdDR group limit
    nonprod_quota_used_vcpu:   Int          ← NonProd subscription vCPU used
    dr_quota_used_vcpu:        Int          ← DR subscription vCPU used
    nonprod_dr_group_used_vcpu: Int          ←  $nonprod\_used + dr\_used$ 
    dr_floor_vcpu:             Int          ←  $potential\_dr\_demand \times vCPU \times dr\_ratio\_max$ 
    effective_nonprod_ceiling_vcpu: Int      ←  $nonprod\_dr\_group\_limit - dr\_floor\_vcpu$ 
    nonprod_headroom_vcpu:     Int          ←  $effective\_nonprod\_ceiling - non-$ 
prod_quota_used
    dr_headroom_vcpu:         Int          ←  $nonprod\_dr\_group\_limit - non-$ 
prod_dr_group_used
    dr_floor_compliant:       Boolean      ←  $dr\_quota\_used \leq dr\_floor\_vcpu$ 

    // — LEGACY FIELDS [Superseded for scoring — retained for per-subscription monit-
oring] —
    quota_limit:             Int          ← [Legacy]  $\sum QuotaRecord.quota\_limit$  (SKU
family)
    quota_used:              Int          ← [Legacy]  $\sum QuotaRecord.quota\_used$ 
    committed_by_crs:        Int          ← [Legacy]  $\sum QuotaRecord.committed\_by\_crs$ 
    deployment_headroom:     Int          ← [Legacy]  $quota\_limit - quota\_used - com-$ 
mitted_by_crs

    NOTE: superseded by

    prod_group_headroom_vcpu /

    nonprod_headroom_vcpu / dr_headroom_vcpu

    for scoring.

    Retain for per-subscription audit trail
    only.

    // — PER-CRG-TYPE FIELDS [NEW — D9] — required by PS_Prod, PS_NonProd, PS_DR,
    HC-6 —
    // Populated by the Quota Sync Worker (E03-S10) from the CRG and Quota Group en-
    tities.

    // Prod CRG
    prod_crg_id:             String       ← ARM resource ID of Prod CRG in this re-
    gion
    prod_crg_quantity:       Int          ← total reserved quantity (sum of all CRs
    in Prod CRG)
    prod_crg_allocated:      Int          ← VMs currently associated with Prod CR
    prod_crg_free_slots:     Int          ←  $prod\_crg\_quantity - prod\_crg\_allocated$ 
    prod_crg_quota_headroom_vcpu: Int      ←  $prod\_group\_headroom\_vcpu$  (aliased for
    Prod CRG context)
    prod_crg_quota_limit_vcpu: Int          ←  $prod\_group\_limit\_vcpu$  (aliased for Prod
    CRG context)

    // NonProd CRG

```

```

    nonprod_crg_id:      String  ← ARM resource ID of NonProd CRG in this
region
    nonprod_crg_quantity: Int    ← total reserved quantity in NonProd CRG
    nonprod_crg_allocated: Int   ← VMs currently associated with NonProd
CR
    nonprod_crg_free_slots: Int  ← nonprod_crg_quantity - non-
prod_crg_allocated
    nonprod_crg_dr_overflow_reserve: Int ← slots reserved for DR overflow (engine-
managed)

                                Set by the engine as a soft ceiling to
                                protect DR headroom.
                                Default: prod_vm_count × dr_ratio_min
                                range, to ensure minimum DR headroom al-
ways exists).
    nonprod_crg_effective_free: Int ← nonprod_crg_free_slots - non-
prod_crg_dr_overflow_reserve

                                The capacity truly available to NonProd
                                workloads.
                                Used in: PS_Prod α, PS_NonProd α,
                                PS_NonProd δ, HC-6.
    nonprod_crg_quota_headroom_vcpu: Int ← nonprod_headroom_vcpu (aliased for Non-
Prod CRG context)
    nonprod_crg_quota_limit_vcpu: Int   ← effective_nonprod_ceiling_vcpu (aliased
for NonProd CRG)

    // DR CRG
    dr_crg_id:      String  ← ARM resource ID of DR CRG in this re-
region
    dr_crg_quantity: Int    ← total reserved quantity in DR CRG
    dr_crg_allocated: Int   ← VMs/VMSS currently associated with DR
CR
    dr_crg_free_slots: Int  ← dr_crg_quantity - dr_crg_allocated
    dr_crg_quota_headroom_vcpu: Int ← dr_headroom_vcpu (aliased for DR CRG
context)
    dr_crg_quota_limit_vcpu: Int   ← dr_floor_vcpu (aliased for DR CRG con-
text)
    potential_dr_demand: Int       ← Σ prod_crg_allocated for all customers
where

                                dr_region == this region. Used as
                                coverage denominator.
    dr_crg_coverage_ratio: Float   ← dr_crg_quantity / potential_dr_demand
Measures how well the DR CRG covers
total potential demand.

                                Null when potential_dr_demand == 0 (no
                                customers yet).
                                Range target: [dr_ratio_min,
                                dr_ratio_max] = [0.30, 0.40].
                                Used in: PS_Prod δ, PS_DR δ, HC-6 (via
                                coverage floor check).

    // — DR AND DISTRIBUTION FIELDS —
    dr_buffer_compliant: Boolean ← all DRCapacityPairs in region at buffer %
    consumer_count_prod: Int    ← customers with Prod in this region
    consumer_count_nonprod: Int ← customers with NonProd in this region
    consumer_count_dr: Int      ← customers with DR in this region
    total_customers: Int        ← all customers across all env types
    az_count: Int               ← distinct physical AZs with capacity
    avg_crg_weight: Float       ← computed CRG weight (see below)
    last_updated: DateTime
}

```

Redis cache key mapping:

quota:group:{region}:prod	→ Prod group headroom (scalar, fast path for HC-3)
quota:group:{region}:nonprod_dr	→ Full NonProdDR state (JSON blob, all derived fields)
snapshot:{region}	→ Full RegionalSnapshot JSON

CRG Weight

Each CRG contributes a weight reflecting how available it is for new customer placements. The **per-CRG weight** is:

```
CRGWeight(crg) =
  (1 - crg.utilization_pct)           ← free capacity ratio
[0..1]
  × ((100 - crg.current_consumer_count) / 100)  ← sharing headroom (100-
consumer limit)
  × purpose_multiplier(crg.purpose)             ← Primary=1.0, Burst=0.7,
DR=0.3, Test=0.0
```

The **Regional CRG Weight (RCW)** is the average across all CRGs in the region:

```
RCW(region) = mean( CRGWeight(crg) for all crgs in region )
```

A region with all CRGs at 90% allocated, 80+ consumers, or DR/Test purpose will have a low RCW and rank poorly for new Prod/NonProd placements.

The Placement Scoring Formula [UPDATED — see D9]

The engine uses **three environment-type-specific Placement Score formulas** — one per environment class. Each formula applies the same five component weights (α through ϵ), but the meaning of each component is specific to the environment type being scored. This replaces the earlier single generic $PS(r, E)$ formula (retained at the end of this section for reference).

PS_Prod(r) is dual-purpose: used by the engine to recommend a Prod region to the customer and to validate a customer-selected Prod region before assignment.

PS_NonProd(r) and **PS_DR(r)** are used by the sequential selection algorithm (Steps 2 and 3 respectively).

All three formulas require per-CRG-type fields from the `RegionalSnapshot` (see Regional State Model below).

DR Ratio Parameters [NEW — see D9]

```

dr_ratio_min    = 0.30    ← lower bound of acceptable DR coverage ratio; used as HC-6
floor
dr_ratio_max    = 0.40    ← upper bound / target ceiling; engine always optimises to-
ward this value
dr_ratio_target = [dr_ratio_min, dr_ratio_max] = [0.30, 0.40]
                                ← acceptable range; engine targets dr_ratio_max as the de-
sired state
                                ← fixing the ceiling at 0.40 ensures higher buffer margins
are always preferred

```

Both `dr_ratio_min` and `dr_ratio_max` are stored in `PlacementPolicy.rules`. The DR floor formula always uses `dr_ratio_max` to prevent undersizing (see Decision D7).

Env-Type-Specific Placement Score Formulas [NEW — see D9]

All sub-scores are normalized to **[0.0, 1.0]** and all five weights sum to 1.0.

PS_Prod(r) — Production Region Score

```

PS_Prod(r) =
  0.30 × (nonprod_crg_effective_free(r) / prod_crg_quantity(r))           ← α:
NonProd overflow headroom
+ 0.20 × (prod_crg_quota_headroom_vcpu(r) / prod_crg_quota_limit_vcpu(r)) ← β:
Prod quota headroom ratio
+ 0.25 × (1 - consumer_count_prod(r) / total_customers)                 ← γ: Prod
distribution fairness
+ 0.15 × dr_crg_coverage_ratio(r)                                         ← δ: DR
CRG coverage readiness
+ 0.10 × (az_count(r) / 3)                                                ← ε: zone
diversity

```

Component semantics:

- **α**: NonProd CRG effective_free (after DR overflow reserve) relative to Prod CRG size. High α means the associated NonProd CRG has headroom to absorb overflow if Prod grows.
- **δ**: `dr_crg_coverage_ratio` of this region's own DR CRG. When scoring a Prod region, δ captures the health of the region's DR CRG — a Prod region whose DR CRG is well-covered scores higher.

Guards:

```

if prod_crg_quantity == 0          → PS_Prod = 0.0
if prod_crg_quota_limit_vcpu == 0  → β term = 0.0
if dr_crg_coverage_ratio is None   → δ term = 0.0 (no DR CRG established yet)

```

PS_NonProd(r) — Non-Production Region Score

```

PS_NonProd(r) =
  0.30 × (nonprod_crg_effective_free(r) / nonprod_crg_quantity(r))           ← α:
effective NonProd headroom
  + 0.20 × (nonprod_crg_quota_headroom_vcpu(r) / nonprod_crg_quota_limit_vcpu(r)) ← β:
NonProd quota headroom
  + 0.25 × (1 - consumer_count_nonprod(r) / total_customers)                 ← γ:
NonProd distribution
  + 0.15 × (nonprod_crg_effective_free(r) / nonprod_crg_quantity(r))           ← δ:
overflow capacity health
  + 0.10 × (az_count(r) / 3)                                                  ← ε:
zone diversity

```

Component semantics:

- **α**: Fraction of NonProd CRG that is truly available (free_slots minus dr_overflow_reserve). This is the primary capacity signal for NonProd placement.
- **δ**: Shares the same ratio as α — intentionally. It rewards regions that are genuinely under-utilised from both a NonProd-placement and a DR-overflow perspective.

Guards:

```

if nonprod_crg_quantity == 0           → PS_NonProd = 0.0
if nonprod_crg_quota_limit_vcpu == 0   → β term = 0.0

```

PS_DR(r) — Disaster Recovery Region Score

```

PS_DR(r) =
  0.30 × (dr_crg_free_slots(r) / dr_crg_quantity(r))           ← α:
DR CRG headroom
  + 0.20 × (dr_crg_quota_headroom_vcpu(r) / dr_crg_quota_limit_vcpu(r)) ← β:
DR quota headroom
  + 0.25 × (1 - consumer_count_dr(r) / total_customers)         ← γ: DR
distribution
  + 0.15 × min(1.0, dr_crg_coverage_ratio(r) / dr_ratio_max)    ← δ:
coverage ratio health
  + 0.10 × (az_count(r) / 3)                                     ← ε:
zone diversity

```

Component semantics:

- **α**: Fraction of DR CRG capacity that is uncommitted. High α = more room for additional DR assignments.
- **δ**: $\min(1.0, \text{dr_crg_coverage_ratio} / \text{dr_ratio_max})$. Scores 1.0 when coverage ≥ 0.40 (fully at target). Scores proportionally below 1.0 for coverage between 0.30 and 0.40 (e.g. $0.35 / 0.40 = 0.875$). The engine targets `dr_ratio_max` as desired state — coverage above `dr_ratio_min` but below `dr_ratio_max` is acceptable but penalised to drive higher buffer margins.

Guards:

```

if dr_crg_quantity == 0          → PS_DR = 0.0  (no DR CRG – region ineligible)
if dr_crg_quota_limit_vcpu == 0 → β term = 0.0
if dr_crg_coverage_ratio is None → δ term = 0.0  (no demand yet – no coverage to score)

```

Per-Env-Type Weight Summary

All three formulas use the same default weights: $\alpha=0.30$, $\beta=0.20$, $\gamma=0.25$, $\delta=0.15$, $\epsilon=0.10$.

Component	Symbol	Default	PS_Prod	PS_NonProd	PS_DR
CRG Capacity	α	0.30	non-prod_crg_effective_free / prod_crg_quantity	non-prod_crg_effective_free / non-prod_crg_quantity	dr_crg_free_slots / dr_crg_quantity
Quota Headroom	β	0.20	Prod group headroom ratio	NonProd floor-adjusted headroom ratio	DR quota headroom within floor
Distribution	γ	0.25	Prod region fairness	NonProd region fairness	DR region fairness
DR/Overflow Health	δ	0.15	dr_crg_coverage_ratio	non-prod_crg_effective_free / non-prod_crg_quantity	min(1.0, coverage_ratio / dr_ratio_max)
Zone Diversity	ϵ	0.10	az_count / 3	az_count / 3	az_count / 3

Weights are stored in `PlacementPolicy.rules` and are configurable per policy (FR-6, E07-S07).

Legacy Reference Formula [Superseded by D9 — retained for reference]

The prior single generic formula is preserved below. It has been fully superseded by `PS_Prod / PS_NonProd / PS_DR` above. Existing worked examples that reference this formula will be updated in a subsequent pass.

[SUPERSEDED]

$$\begin{aligned} \text{PS}(r, E) = & \alpha \times \text{CRG_Score}(r) \\ & + \beta \times \text{Quota_Score}(r) \\ & + \gamma \times \text{Distribution_Score}(r, E) \\ & + \delta \times \text{DR_Buffer_Score}(r, E) \\ & + \epsilon \times \text{Zone_Score}(r) \end{aligned}$$

Where: $E \in \{\text{NonProd}, \text{DR}\}$ — Prod was previously unscored
 $\alpha=0.30, \beta=0.20, \gamma=0.25, \delta=0.15, \epsilon=0.10$ (defaults)

Sub-Score Definitions

CRG_Score(r) — Regional CRG Weight (Monitoring / Legacy)

`CRG_Score` is now retained for **monitoring and capacity health signalling** only. It is not a direct input to `PS_Prod`, `PS_NonProd`, or `PS_DR` — those use per-CRG-type fields (see Per-Env-Type Weight Summary above). `CRG_Score` continues to be computed in the reconciliation loop as a regional health indicator and audit signal.

$$\begin{aligned} \text{CRG_Score}(r) &= \text{RCW}(r) \\ &= \text{mean}(\text{CRGWeight}(\text{crg}) \text{ for } \text{crg} \text{ in } \text{region}(r)) \end{aligned}$$

where:

$$\begin{aligned} \text{CRGWeight}(\text{crg}) &= (1 - \text{crg.utilization_pct}) \\ &\quad \times ((100 - \text{crg.consumer_count}) / 100) \\ &\quad \times \text{purpose_multiplier}(\text{crg.purpose}) \end{aligned}$$

Interpretation: A region where all CRGs are lightly loaded, have many sharing slots remaining, and are primarily configured as Primary purpose scores close to 1.0. A saturated region scores close to 0.0.

Guard: `if total_crg_count(r) == 0 → CRG_Score = 0.0` (region has no managed CRGs, cannot place).

Quota_Score(r, env_type) — Quota Group Headroom Ratio [UPDATED]

The `Quota_Score` is now **environment-type-aware** — it reads from the correct quota group for the placement being scored. The three env-type formulas are:

```
// For Prod placement (reads from Prod Quota Group):
Quota_Score_Prod(r) = prod_group_headroom_vcpu(r) / prod_group_limit_vcpu(r)

// For NonProd placement (reads from NonProdDR group – floor-adjusted ceiling):
Quota_Score_NonProd(r) = nonprod_headroom_vcpu(r) / effective_nonprod_ceiling_vcpu(r)
  ← uses floor-adjusted ceiling, NOT raw nonprod_dr_group_limit_vcpu
  ← this ensures the score reflects quota truly available to NonProd, not DR-protected quota

// For DR placement (reads from NonProdDR group – DR floor buffer):
Quota_Score_DR(r) = dr_headroom_vcpu(r) / dr_floor_vcpu(r)
  ← measures how much of the DR quota floor remains available
  ← score of 1.0 = DR CRG at zero (maximum expansion room)
  ← score of 0.0 = DR CRG consuming the full floor allocation
```

Note on DR score semantics: A low `Quota_Score_DR` means the DR CRG is large and well-provisioned — good for coverage but signals less expansion room. This tension is intentionally managed: the `DR_Buffer_Score` (δ in `PS_DR`) rewards coverage; `Quota_Score_DR` (β) signals remaining headroom. Together they produce a balanced DR region ranking.

Legacy field note: `deployment_headroom` (per-subscription `QuotaRecord`) is no longer used in the scoring formula. It is retained in the `RegionalSnapshot` for per-subscription audit and monitoring via E03-S01 (legacy path). All HC-3, HC-7, and `Quota_Score` calculations read from the group-level fields.

Guards:

```
if prod_group_limit_vcpu == 0      → Quota_Score_Prod = 0.0
if effective_nonprod_ceiling == 0 → Quota_Score_NonProd = 0.0
if dr_floor_vcpu == 0             → Quota_Score_DR = 1.0  (no floor configured – no
restriction)
```

Scoring pseudocode update: The `compute_quota_score(snap)` call in the selection algorithm becomes `compute_quota_score(snap, env_type)`, switching on `env_type` to return the correct formula. The `apply_hard_constraints` function similarly passes `env_type` to the HC-3/HC-7 evaluators.

Distribution_Score(r, E) — Fair Distribution Across Regions

This is the **load-balancing sub-score**. It rewards regions with fewer customers already assigned to environment type E.

```
Distribution_Score(r, E) = 1.0 - (customer_count(r, E) / total_customers)
```

where:

```
customer_count(r, NonProd) = regional_snapshot.consumer_count_nonprod
customer_count(r, DR)      = regional_snapshot.consumer_count_dr
total_customers            = total active customers across all regions
```

Interpretation: If Region-B already hosts NonProd for 40% of all customers, its `Distribution_Score` for NonProd is 0.60. A fresh region with zero NonProd assignments scores 1.0.

Tiebreaker (Jitter): When two regions produce identical scores (common during bootstrap), add a small uniform random jitter $U(-0.01, +0.01)$ to prevent deterministic concentration. Log the jitter value in the `OperationRecord`.

DR_Buffer_Score(r, E) — DR Capacity Readiness

This sub-score is **environment-aware**:

- For `E = NonProd`: `DR_Buffer_Score` measures whether placing a NonProd environment here still preserves the DR buffer for existing DR pairs that use this region as their DR target.

```

DR_Buffer_Score(r, NonProd) =
  fraction of DRCapacityPairs using region(r) as DR target
  that remain buffer-compliant after absorbing NonProd load

= count(dr_pairs where dr_region==r AND dr_buffer_compliant AFTER reservation)
  / count(dr_pairs where dr_region==r)

If no DR pairs use this region as DR target: DR_Buffer_Score = 1.0 (no impact)

```

- For `E = DR`: `DR_Buffer_Score` measures how well the region can absorb the required DR pre-positioned capacity.

```

DR_Buffer_Score(r, DR) =
  min(1.0, dr_free_slots(r) / required_dr_quantity)

where:
  required_dr_quantity = prod_vm_count × dr_buffer_pct
  dr_free_slots(r)     = total_free_slots in region(r) excluding slots already
  committed to DR pairs

```

Zone_Score(r) — Physical Availability Zone Diversity

```

Zone_Score(r) = min(1.0, az_count(r) / target_az_count)

where:
  az_count(r)      = distinct physical AZs in region(r) that have managed CRs
  target_az_count = 3 (configurable in PlacementPolicy; 3-zone spread is the baseline
  for HA)

```

A region where managed capacity spans all 3 AZs scores 1.0. A region where capacity exists only in 1 AZ scores 0.33.

Selection Algorithm

The algorithm is **sequential**: Prod is given; NonProd is scored from the remaining regions; DR is scored from the remaining regions after NonProd is fixed.

Pseudocode

```

function SELECT_REGIONS(customer_id, prod_region, env_config, policy):

    all_regions = engine.get_managed_regions()          # e.g. {R1, R2, R3} or
{R1,R2,R3,R4}
    snapshots    = redis.get_regional_snapshots()

    # — STEP 1: Prod is fixed —————
    prod = prod_region

    # — STEP 2: Select NonProd —————
    candidates_np = all_regions \ {prod}

    eligible_np = apply_hard_constraints(candidates_np, "NonProd", env_config, snap-
shots)

    if len(eligible_np) == 0:
        raise RegionExhaustedError("No eligible region for NonProd")

    scores_np = {
        r: compute_placement_score(r, "NonProd", snapshots, policy)
        for r in eligible_np
    }
    non_prod = argmax(scores_np)          # highest score wins

    # — STEP 3: Select DR —————
    # [UPDATED D8] DR may share the NonProd region – only Prod is excluded from DR can-
didates.
    candidates_dr = all_regions \ {prod}

    eligible_dr = apply_hard_constraints(candidates_dr, "DR", env_config, snapshots)

    if len(eligible_dr) == 0:
        raise RegionExhaustedError("No eligible region for DR")

    scores_dr = {
        r: compute_placement_score(r, "DR", snapshots, policy)
        for r in eligible_dr
    }
    dr = argmax(scores_dr)

    # — STEP 4: Record and return —————
    assignment = CustomerRegionAssignment {
        customer_id:    customer_id,
        prod_region:    prod,
        nonprod_region: non_prod,
        dr_region:      dr,
        scores:         {non_prod: scores_np, dr: scores_dr}, # audit trail
        policy_id:      policy.policy_id,
        timestamp:      now()
    }
    engine.persist_operation_record(assignment)
    engine.update_regional_snapshots(assignment) # increment consumer counts

    return assignment

function compute_placement_score(r, env_type, snapshots, policy):
    snap = snapshots[r]
     $\alpha$ ,  $\beta$ ,  $\gamma$ ,  $\delta$ ,  $\epsilon$  = policy.weights # from PlacementPolicy.rules

    # [UPDATED D9] Dispatch to env-type-specific formula – PS_Prod / PS_NonProd / PS_DR
    if env_type == "Prod":

```

```

    score = compute_ps_prod(snap,  $\alpha$ ,  $\beta$ ,  $\gamma$ ,  $\delta$ ,  $\epsilon$ )
elif env_type == "NonProd":
    score = compute_ps_nonprod(snap,  $\alpha$ ,  $\beta$ ,  $\gamma$ ,  $\delta$ ,  $\epsilon$ )
elif env_type == "DR":
    score = compute_ps_dr(snap,  $\alpha$ ,  $\beta$ ,  $\gamma$ ,  $\delta$ ,  $\epsilon$ , policy.dr_ratio_max)
else:
    raise ValueError("Unknown env_type: " + env_type)

score += uniform_jitter(-0.01, +0.01) # tiebreaker
return score

# [NEW D9] PS_Prod – per-CRG-type fields required in snap
function compute_ps_prod(snap,  $\alpha$ ,  $\beta$ ,  $\gamma$ ,  $\delta$ ,  $\epsilon$ ):
    if snap.prod_crg_quantity == 0: return 0.0
    a_score = snap.nonprod_crg_effective_free / snap.prod_crg_quantity
    b_score = snap.prod_crg_quota_headroom_vcpu / snap.prod_crg_quota_limit_vcpu if
snap.prod_crg_quota_limit_vcpu > 0 else 0.0
    g_score = 1.0 - (snap.consumer_count_prod / snap.total_customers) if
snap.total_customers > 0 else 1.0
    d_score = snap.dr_crg_coverage_ratio if snap.dr_crg_coverage_ratio is not None else
0.0
    e_score = min(1.0, snap.az_count / 3)
    return  $\alpha$ *a_score +  $\beta$ *b_score +  $\gamma$ *g_score +  $\delta$ *d_score +  $\epsilon$ *e_score

# [NEW D9] PS_NonProd – per-CRG-type fields required in snap
function compute_ps_nonprod(snap,  $\alpha$ ,  $\beta$ ,  $\gamma$ ,  $\delta$ ,  $\epsilon$ ):
    if snap.nonprod_crg_quantity == 0: return 0.0
    a_score = snap.nonprod_crg_effective_free / snap.nonprod_crg_quantity
    b_score = snap.nonprod_crg_quota_headroom_vcpu / snap.nonprod_crg_quota_limit_vcpu
if snap.nonprod_crg_quota_limit_vcpu > 0 else 0.0
    g_score = 1.0 - (snap.consumer_count_nonprod / snap.total_customers) if
snap.total_customers > 0 else 1.0
    d_score = a_score # same ratio – intentional (overflow health mirrors NonProd
headroom)
    e_score = min(1.0, snap.az_count / 3)
    return  $\alpha$ *a_score +  $\beta$ *b_score +  $\gamma$ *g_score +  $\delta$ *d_score +  $\epsilon$ *e_score

# [NEW D9] PS_DR – per-CRG-type fields required in snap; dr_ratio_max from policy
function compute_ps_dr(snap,  $\alpha$ ,  $\beta$ ,  $\gamma$ ,  $\delta$ ,  $\epsilon$ , dr_ratio_max):
    if snap.dr_crg_quantity == 0: return 0.0
    a_score = snap.dr_crg_free_slots / snap.dr_crg_quantity
    b_score = snap.dr_crg_quota_headroom_vcpu / snap.dr_crg_quota_limit_vcpu if
snap.dr_crg_quota_limit_vcpu > 0 else 0.0
    g_score = 1.0 - (snap.consumer_count_dr / snap.total_customers) if
snap.total_customers > 0 else 1.0
    d_score = min(1.0, snap.dr_crg_coverage_ratio / dr_ratio_max) if
snap.dr_crg_coverage_ratio is not None else 0.0
    e_score = min(1.0, snap.az_count / 3)
    return  $\alpha$ *a_score +  $\beta$ *b_score +  $\gamma$ *g_score +  $\delta$ *d_score +  $\epsilon$ *e_score

# [UPDATED] Quota score dispatch – routes to correct group formula by env_type
function compute_quota_score(snap, env_type):
    if env_type == "Prod":
        if snap.prod_group_limit_vcpu == 0: return 0.0
        return snap.prod_group_headroom_vcpu / snap.prod_group_limit_vcpu

    elif env_type == "NonProd":
        if snap.effective_nonprod_ceiling_vcpu == 0: return 0.0
        return snap.nonprod_headroom_vcpu / snap.effective_nonprod_ceiling_vcpu

    elif env_type == "DR":
        if snap.dr_floor_vcpu == 0: return 1.0 # no floor configured
        return snap.dr_headroom_vcpu / snap.dr_floor_vcpu

```

```

else:
    raise ValueError("Unknown env_type: " + env_type)

# [UPDATED] Hard constraint check – HC-3 and HC-7 are now env_type-aware
function apply_hard_constraints(candidates, env_type, env_config, snapshots):
    eligible = []
    for r in candidates:
        snap = snapshots[r]
        vm_count = env_config.vm_count
        vcpu_count = env_config.vcpu_per_instance

        # HC-1: region separation – already applied by caller (candidates list excludes
        # used regions)
        # HC-2: capacity floor
        if snap.total_free_slots < env_config.min_reservation_units:
            continue
        # HC-3 [UPDATED]: Quota Group headroom check
        if env_type == "Prod":
            if snap.prod_group_headroom_vcpu < vm_count * vcpu_count:
                continue
        elif env_type == "NonProd":
            if snap.nonprod_headroom_vcpu < vm_count * vcpu_count:
                continue
        elif env_type == "DR":
            target_dr_vcpu = env_config.prod_vm_count * env_config.dr_ratio * vcpu_count
            if snap.dr_headroom_vcpu < target_dr_vcpu:
                continue
        # HC-4: separation class (enforced externally by region set composition)
        # HC-5: zone availability
        if snap.az_count < 2:
            continue
        # HC-6 [NEW D8]: DR coverage floor – DR only
        # Combined DR CRG free slots + NonProd CRG effective_free must cover customer's DR
        # demand
        if env_type == "DR":
            customer_dr_slots = env_config.prod_vm_count * env_config.dr_ratio_max
            combined_dr_cap = snap.dr_crg_free_slots + snap.nonprod_crg_effective_free
            if combined_dr_cap < customer_dr_slots:
                continue # insufficient combined DR capacity in this region

        # HC-7 [NEW]: DR floor integrity – NonProd only
        if env_type == "NonProd":
            projected_nonprod_vcpu = snap.nonprod_quota_used_vcpu + (vm_count * vcpu_count)
            if projected_nonprod_vcpu > snap.effective_nonprod_ceiling_vcpu:
                continue # would encroach on DR floor

        eligible.append(r)
    return eligible

```

Time Complexity

$O(R \times C)$ where R = number of candidate regions (3-4) and C = average CRG count per region. With $R=4$ and $C=20$ CRGs, this is 80 weight computations — sub-millisecond on cached data.

Worked Examples

Example A — 3 Regions:

State at time T:

Region	total_quantity	total_allocated	nonprod_customers	dr_customers	az_count
R1	200	120	3	2	3
R2	200	60	1	1	3
R3	200	40	0	0	3

Quota headroom and DR buffer all pass HC-2/HC-3. Customer-X selects Prod = R1.

NonProd scoring (R1 excluded):

	CRG_Score	Quota_Score	Distribution_Score	DR_Buffer	Zone
→ PS					
R2 (WestEurope)	0.70	0.70	$1 - 1/4 = 0.75$	1.0	1.0
→ $0.30 \times 0.70 + 0.20 \times 0.70 + 0.25 \times 0.75 + 0.15 \times 1.0 + 0.10 \times 1.0 = 0.21 + 0.14 + 0.19 + 0.15 + 0.10 = 0.79$					
R3 (SoutheastAsia)	0.80	0.80	$1 - 0/4 = 1.00$	1.0	1.0
→ $0.30 \times 0.80 + 0.20 \times 0.80 + 0.25 \times 1.00 + 0.15 \times 1.0 + 0.10 \times 1.0 = 0.24 + 0.16 + 0.25 + 0.15 + 0.10 = 0.90 \checkmark$					

NonProd → R3 (SoutheastAsia) wins (less loaded, higher headroom).

DR scoring (R1, R3 excluded) → only R2 remains:

→ **DR → R2 (WestEurope)** (no competitor; confirm HC passes).

Final Assignment: Prod=R1, NonProd=R3, DR=R2.

Example B — 4 Regions:

State at time T (later; R3 is now more loaded):

Region	total_quantity	total_allocated	nonprod_customers	dr_customers	az_count
R1	200	150	5	3	3
R2	200	100	3	4	3
R3	200	160	4	2	3
R4	200	80	2	1	3

Customer-Y selects Prod = R2.

NonProd scoring (R2 excluded — R1, R3, R4 eligible):

	CRG_Score	Quota_Score	Dist_Score(NonProd)	DR_Buffer	Zone	→ PS
R1(EastUS)	0.25	0.25	1 - 5/14 = 0.64	1.0	1.0	→
$0.30 \times 0.25 + 0.20 \times 0.25 + 0.25 \times 0.64 + 0.15 \times 1.0 + 0.10 \times 1.0 = 0.075 + 0.050 + 0.160 + 0.150 + 0.100 = 0.535$						
R3(SEAsia)	0.20	0.25	1 - 4/14 = 0.71	0.9	1.0	→
$0.30 \times 0.20 + 0.20 \times 0.25 + 0.25 \times 0.71 + 0.15 \times 0.90 + 0.10 \times 1.0 = 0.060 + 0.050 + 0.178 + 0.135 + 0.100 = 0.523$						
R4(AusEast)	0.60	0.60	1 - 2/14 = 0.86	1.0	1.0	→
$0.30 \times 0.60 + 0.20 \times 0.60 + 0.25 \times 0.86 + 0.15 \times 1.0 + 0.10 \times 1.0 = 0.180 + 0.120 + 0.215 + 0.150 + 0.100 = 0.765$ ✓						

NonProd → R4 (AustraliaEast) wins (most headroom, least concentrated).

DR scoring (R2, R4 excluded — R1, R3 eligible):

	CRG_Score	Quota_Score	Dist_Score(DR)	DR_Buffer	Zone	→ PS
R1(EastUS)	0.25	0.25	1 - 3/14 = 0.79	1.0	1.0	→
$0.075 + 0.050 + 0.196 + 0.150 + 0.100 = 0.571$						
R3(SEAsia)	0.20	0.25	1 - 2/14 = 0.86	0.9	1.0	→
$0.060 + 0.050 + 0.214 + 0.135 + 0.100 = 0.559$						

DR → R1 (EastUS) wins (slightly better quota and distribution).

Final Assignment: Prod=R2, NonProd=R4, DR=R1.

Rebalancing and Drift Handling

Natural Convergence

The `Distribution_Score` sub-component (γ) acts as a **self-correcting load balancer** over time. As a region accumulates NonProd customers its `consumer_count_nonprod` rises, lowering its `Distribution_Score` and redirecting future placements away from it — until all regions reach a roughly equal count.

With `total_customers = T` and 3 regions:

- **Steady-state expected count per region per env type = $T/3$**

- Any departure from this ratio is automatically corrected by the formula without operator intervention.

With 4 regions:

- Steady-state = $T/4$ per region per env type.

- In practice, Prod is customer-driven, so NonProd/DR distributions will converge independently.

Distribution Saturation Alert

When all candidate regions for an env type have `Distribution_Score < 0.2` (i.e. all are heavily loaded), emit `RegionDistributionSaturated` event — this signals the need to onboard a new region or expand capacity in an existing one before placement quality degrades.

Capacity Exhaustion Path

When a region fails HC-2 (capacity floor): it is removed from scoring. If all candidates fail HC-2, the placement request fails with `RegionExhaustedError`. The engine should:

1. Alert operators via `CapacityExhaustedAlert`.
2. Automatically trigger quota increase requests (E03-S06) in the saturated region(s).

3. Place the customer in a queue with a `pending_placement` flag and re-evaluate when capacity refreshes (within one reconciliation cycle).

Score Staleness

Regional snapshots have a 5-minute TTL (matching the reconciliation engine cycle). If a snapshot is older than 10 minutes (e.g. reconciliation loop is stuck), placement falls back to read directly from Cosmos DB and emits a `StaleRegionalSnapshot` warning in the operation record. Never block placement on a snapshot failure — degrade gracefully.

Steady State Capacity Lifecycle Management [NEW — see D10]

Note: This section describes the **non-crisis** capacity growth path. It is a completely separate operating system from Emergency Capacity Transfer (next section). The steady-state path fires when coverage ratios decline due to organic Prod growth — it is proactive and approval-gated (Phase A) or self-managed (Phase B). It never disassociates VMs and never sacrifices NonProd SLA.

Auto-Increase Trigger

The reconciliation loop (5-min cycle) evaluates the following condition for each CRG type in each managed region:

```
// DR CRG trigger – primary use case
IF dr_crg_coverage_ratio < policy.dr_autoincrease_threshold
  AND engine_mode == STEADY_STATE           ← not during active DR event
  AND region.last_autoincrease_at < (now - policy.autoincrease_cooldown_minutes)
THEN raise CapacityIncreaseRequest(region, CRG_type=DR)

// Prod CRG trigger
IF (prod_crg_free_slots / prod_crg_quantity) < policy.prod_autoincrease_threshold
  AND engine_mode == STEADY_STATE
  AND region.last_autoincrease_at < (now - policy.autoincrease_cooldown_minutes)
THEN raise CapacityIncreaseRequest(region, CRG_type=Prod)

// NonProd CRG trigger
IF (nonprod_crg_effective_free / nonprod_crg_quantity) <
policy.nonprod_autoincrease_threshold
  AND engine_mode == STEADY_STATE
  AND region.last_autoincrease_at < (now - policy.autoincrease_cooldown_minutes)
THEN raise CapacityIncreaseRequest(region, CRG_type=NonProd)
```

Threshold semantics:

CRG Type	Metric	Default Threshold	Signal
DR	<code>dr_crg_coverage_ratio</code>	0.35	Coverage falling toward HC-6 floor (0.30); acts before placement is blocked
Prod	<code>prod_crg_free_slots / prod_crg_quantity</code>	0.20	CRG 80% full; headroom shrinking
NonProd	<code>non-prod_crg_effective_free / non-prod_crg_quantity</code>	0.20	Effective headroom (after DR floor) 80% consumed

All three thresholds are configurable in `PlacementPolicy.rules`. The HC-6 hard floor (0.30) and HC-7 enforcement remain as safety backstops — the trigger fires proactively above them so HC-6/HC-7 should never be the first signal the engine observes.

Debounce guard: Once a `CapacityIncreaseRequest` is raised for a region+CRG_type pair, no further request is raised for that pair within `autoincrease_cooldown_minutes` (default: 30 min). This prevents rapid-fire requests during volatile demand. A staleness alert (`AutoIncreaseDebounceActive`) is emitted if the metric remains below threshold throughout the cooldown to ensure an operator can intervene manually if needed.

Target quantity calculation (DR CRG):

```
delta_qty    = ceil((dr_ratio_max - dr_crg_coverage_ratio) × potential_dr_demand)
target_qty   = dr_crg_current_qty + delta_qty
              ← brings coverage_ratio back to dr_ratio_max (0.40) – the desired state
```

Target quantity calculation (Prod / NonProd CRG):

```
delta_qty    = ceil(growth_buffer_pct × crg_current_qty)
              ← default: add 20% of current quantity; configurable via policy
target_qty   = crg_current_qty + delta_qty
```

CapacityIncreaseRequest Entity (Cosmos DB)

```
CapacityIncreaseRequest {
  request_id:          GUID (PK)
  region:              String
  crg_type:             Enum [Prod, NonProd, DR]
  current_quantity:    Int
  target_quantity:      Int          ← computed from trigger delta; see above
  trigger_metric:       Float        ← coverage_ratio or free_ratio at trigger time
  trigger_threshold:    Float        ← policy value that was crossed
  quota_increase_needed_vcpu: Int    ← 0 if group headroom sufficient; > 0 if group increase also required
  status:               Enum [PENDING_APPROVAL, APPROVED, EXECUTING, COMPLETED, REJECTED, FAILED]
  requested_by:         String       ← "engine-reconciliation-loop" (Phase A / B auto)
  approved_by:         String?      ← operator GUID (Phase A) or "engine-policy" (Phase B)
  created_at:           DateTime
  approved_at:          DateTime?
  completed_at:         DateTime?
  error:                String?     ← populated on FAILED status
}
```

Phase A — Approval-Gated Workflow (Current)

1. Reconciliation loop detects trigger metric below threshold.
2. Engine creates CapacityIncreaseRequest { status: PENDING_APPROVAL }.
3. Alert raised: CapacityIncreaseRequired (Severity: High)
Payload: region, crg_type, current_metric, threshold, target_qty, quota_increase_needed_vcpu
4. Operator reviews request in ACRME control plane; approves or rejects.
On REJECT: status → REJECTED; alert closed; no ARM operations.
On APPROVE: status → APPROVED → EXECUTING.
5. Engine executes:
 - a. PATCH CR quantity to target_qty.
 - b. IF quota_increase_needed_vcpu > 0:
POST Microsoft.Quota/groupQuotas/{group_id}/quota
Poll until quota limit confirmed updated.
6. RegionalSnapshot updated: crg_quantity, free_slots, coverage_ratio refreshed.
7. status → COMPLETED; alert resolved.
8. OperationRecord written with full audit trail.

Phase B — Self-Managed Automated Workflow (Future)

1. Reconciliation loop detects trigger metric below threshold.
2. Engine evaluates group headroom:
 - CASE headroom sufficient for target expansion:
 - PATCH CR quantity directly (no approval, no quota call).
 - Alert: CapacityAutoIncreaseExecuted (Severity: Info).
 - CASE headroom insufficient (quota_increase_needed_vcpu > 0):
 - POST Microsoft.Quota/groupQuotas/{group_id}/quota.
 - Poll for approval (Azure-side); expand CR once confirmed.
 - Alert: CapacityAutoIncreaseWithQuotaRequest (Severity: Info).
3. All operations logged in OperationRecord with "engine-policy" as approved_by.

Transition from Phase A → Phase B: Single config change — `policy.autoincrease_auto_approve: false → true`. No code change required. Audit trail format is identical between phases.

Emergency Capacity Transfer (Crisis Mode) [NEW — see D10, D11]

Operating boundary: Emergency Capacity Transfer is activated **only during an active DR event** — when the primary Prod region is confirmed down and DR failover VMs need to be deployed. This system is intentionally separate from steady-state capacity growth. Triggering it outside a declared DR event requires operator gate.

Relabelling note: The prior document referred to the quota-neutral NonProd→DR CR transfer as “Tier 3 Emergency Transfer.” Under this model, that operation is correctly classified as **Tier 2 — QuotaNeutralTransfer**. The old label is superseded by D11.

Tier Model

Tier	Name	SLA Impact	Trigger Condition	Approval
Tier 1 fully mated	DirectExpansion None		DR event active AND emergency_ transfer_headroom has available vCPU to cover requested_slots	None — auto-
Tier 2 en: auto gate	QuotaNeutral NonProd CR SLA Transfer removed (VM keeps running, Path B)		Tier 1 insufficient (headroom exhausted) AND NonProd CRG qty > 0 (quantity available to reduce)	Policy-driv- configurable OR confirm
Tier 3 gated: RBAC + proval quired	Destructive NonProd VMs lose Transfer CR SLA; Path B applied (VM stays running)		Tier 2 insufficient OR DR CRG has zero free slots AND DR failover VMs cannot deploy	Operator- elevated dual ap- re-

VM/VMSS disassociation (Tier 3) is only triggered when an entire primary region is down and DR failover VMs cannot deploy. This is the most severe operation in the engine. NonProd workloads are sacrificed to free CR slots in the DR region.

Tier Escalation Logic

```

EmergencyCapacityTransfer(requested_slots, dr_region, customer_id) called
├── Guard: engine_mode must == DR_EVENT_ACTIVE
│   │   (reject if called outside declared DR event – operator error protection)
└── ▼
    [ Tier 1 evaluation ]
    Can emergency_headroom_available_vcpu ≥ requested_slots × vCPU_per_instance?
    YES → Execute Tier 1 (DR CRG quantity expansion using headroom)
        → COMPLETED (RTO: ARM propagation only – minutes)
    NO ↓
    [ Tier 2 evaluation ]
    Can nonprod_crg_quantity reduction cover the deficit?
    (nonprod_crg_quantity > 0 AND freed_vcpu_if_reduced_to_0 ≥ deficit_vcpu)
    YES → Evaluate Tier 2 (policy-gated)
        IF tier2_auto_approve == true: execute directly
        IF tier2_auto_approve == false: → PENDING_APPROVAL; block until operator
        confirms
        → COMPLETED (RTO: approval time + ARM + POC-32 propagation latency)
    NO ↓
    [ Tier 3 evaluation ]
    Operator must provide vm_disassociation_list
    → PENDING_APPROVAL (dual approval required: tier3_dual_approval_required == true)
    → Execute on approval (RTO: approval time + ARM × VM count)
  
```

max_emergency_transfer_qty — Pre-Staged Headroom Formula [NEW — see D10]

The `emergency_transfer_headroom_vcpu` term in the NonProd+DR Group budget is pre-staged to support Tier 1 and Tier 2 crisis operations. It is **separate from** the steady-state auto-increase path, which uses normal group headroom.

```

max_emergency_transfer_qty =
    potential_dr_demand(region) × policy.emergency_transfer_pct

emergency_transfer_headroom_vcpu =
    max_emergency_transfer_qty × vCPU_per_instance
  
```

PlacementPolicy field:

```

"emergency_transfer_pct": 0.30 // fraction of potential_dr_demand to pre-stage as
crisis headroom                // Default 0.30 = 30% of Prod demand additional
                                // crisis capacity
                                // Sizing rationale:
                                //   DR CRG holds 40% of Prod demand (dr_ratio_max)
                                //   Emergency headroom adds 30% → combined crisis
capacity = 70%                  //   Remaining 30% requires Tier 3 VM disassociation
or customer scale-out
  
```

Economic note: This headroom is pre-reserved quota — it is always charged at the group quota budget level regardless of whether a crisis occurs. Operators should tune `emergency_transfer_pct` against RTO risk appetite.

Tier-Level Approval Configuration

Stored in `PlacementPolicy.rules` under `emergency_transfer_approval`:

```
"emergency_transfer_approval": {
  "tier1_auto_approve":      true,                // always automated – no
  override                    // Phase A: approval re-
  "tier2_auto_approve":      false,                quired; Phase B: true
  "tier3_auto_approve":      false,                // always false – enforced
  in code, not config
  "tier2_approver_role":      "ACRME.CapacityAdmin",
  "tier3_approver_role":      "ACRME.EmergencyOperator", // elevated RBAC
  "tier3_dual_approval_required": true,
  "break_glass_role":         "ACRME.SuperAdmin"      // single-approver
  override; always audit-logged
}
```

EmergencyCapacityTransfer API Operation [NEW — see D11]

Method: POST

Endpoint: `/api/v1/capacity/emergency-transfer`

Auth: Role varies by tier (see approval config above). Tier 3 requires `ACRME.EmergencyOperator`.

Request Schema

```
EmergencyCapacityTransferRequest {
  customer_id:      GUID                // target customer
  dr_region:        String               // DR region needing capacity
  transfer_tier:     Enum [Tier1, Tier2, Tier3] // requested tier; engine may
  escalate          // additional DR CR slots
  requested_slots:   Int
  needed
  requestor_id:      GUID                // operator submitting request
  justification:      String              // required for Tier 2 and
  Tier 3 (audit)
  correlation_id:     GUID                // links to DR incident tick-
  et / IncidentRecord

  // Tier 3 only – operator-provided VM disassociation list
  vm_disassociation_list: VM_DisassociationTarget[] // empty for Tier 1 / Tier 2
}

VM_DisassociationTarget {
  vm_id:      GUID
  vm_name:     String
  subscription_id: GUID                // must be a NonProd subscrip-
  tion (validated)
  resource_group: String
  nonprod_region: String
  disassociation_path: Enum [PathA, PathB] // PathB is default (VM keeps
  running, loses SLA)
  environment_tier: Enum [Dev, Test, Staging] // operator-declared; used for
  ordering + audit
}
```

Response Schema

```

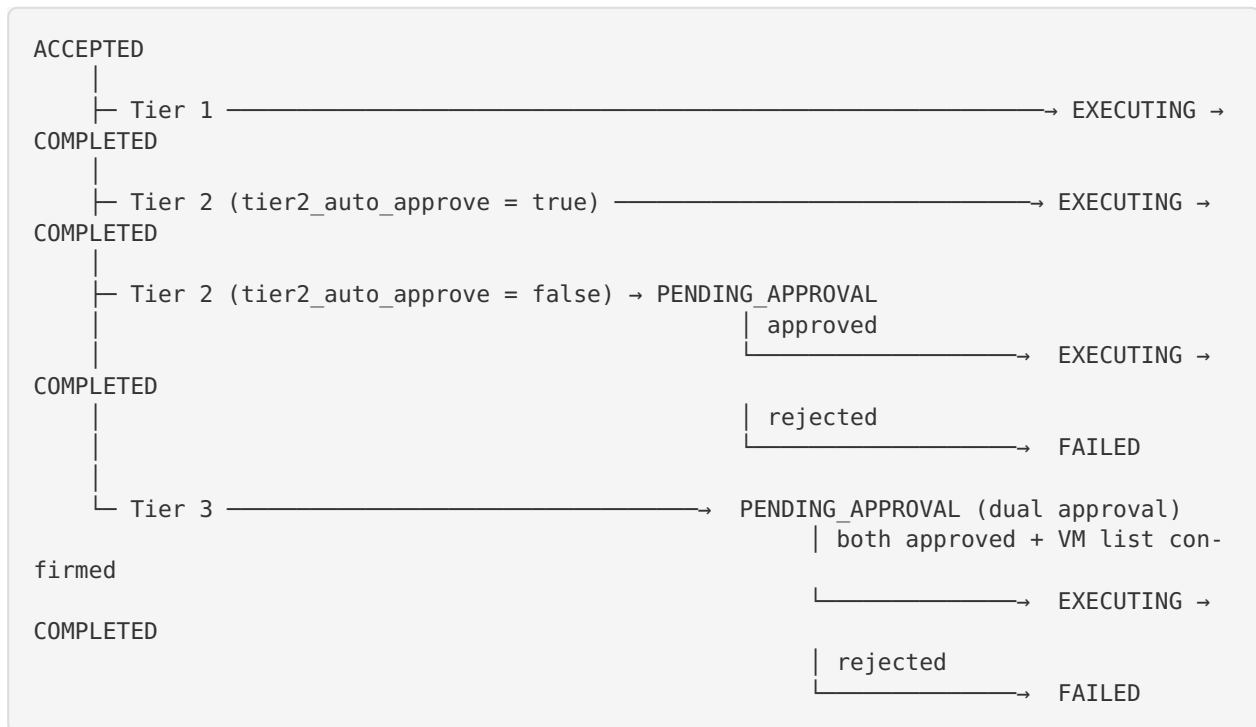
EmergencyCapacityTransferResponse {
  operation_id:          GUID
  status:                Enum [ACCEPTED, PENDING_APPROVAL, EXECUTING, COMPLETED,
FAILED, ESCALATED]
  tier_evaluated:        Enum [Tier1, Tier2, Tier3] // tier the engine actually
used
  tier_escalated:        Boolean                    // true if engine escalated
beyond requested tier
  quota_neutral:         Boolean                    // true for Tier 1 and Tier 2
  slots_requested:       Int
  slots_transferred:     Int                        // 0 if pending approval
  estimated_rto_minutes: Int                        // 0=Tier1; propaga-
tion_latency+ARM=Tier2; longer=Tier3
  execution_steps:       ExecutionStep[]           // ARM operation audit trail
  vm_impact_log:         VM_ImpactRecord[]         // Tier 3 only; empty for
Tier 1 / Tier 2
  created_at:            DateTime
  approved_at:           DateTime?
  completed_at:          DateTime?
}

ExecutionStep {
  step_number:           Int
  step_type:             Enum [CR_QUANTITY_REDUCE, CR_QUANTITY_EXPAND,
QUOTA_INCREASE,
                        VM_DISASSOCIATE, QUOTA_PROPAGATION_WAIT]
  resource_id:           String                    // ARM resource URI
  status:                Enum [PENDING, EXECUTING, COMPLETED, FAILED]
  started_at:            DateTime?
  completed_at:          DateTime?
  error:                 String?
}

VM_ImpactRecord {
  vm_id:                 GUID
  vm_name:               String
  disassociation_path:   Enum [PathA, PathB]
  status:                Enum [PENDING, DISASSOCIATED, FAILED]
  sla_impact:            String                    // "CR SLA removed – VM
continues running (PathB)"
  completed_at:          DateTime?
}

```


Operation State Machine



VM/VMSS Disassociation Sequence — Tier 3 Only [NEW — see D11]

Preconditions (engine validates before execution)

1. All VM IDs in `vm_disassociation_list` confirmed in NonProd subscriptions.
Prod VMs are **INELIGIBLE** – enforced in code; engine rejects list if any Prod VM included.
2. All VMs confirmed associated with the NonProd CRG in target `nonprod_region` (ARM GET validation).
3. $\Sigma(\text{freed_slots_from_vm_list}) \geq \text{requested_slots}$ – list must cover the full capacity demand.
4. Tier 3 dual approval confirmed in engine state before any ARM operations begin.

Execution Ordering

VMs are processed in order of `environment_tier`: **Dev → Test → Staging** (lowest-criticality first). Within each tier, VMs with the smallest vCPU count are processed first (finest-grained capacity release per ARM call). This order minimises blast radius — often Dev VMs alone are sufficient to cover the deficit, leaving Test and Staging untouched.

Execution Sequence (Path B Default)

```

FOR EACH vm IN vm_disassociation_list
  (ordered: Dev first → Test → Staging; within tier: smallest vCPU first):

  — STEP 1: Reduce Provider CR quantity to 0 (Path B – Provider side) —
  PUT /subscriptions/{provider_sub}/resourceGroups/{rg}/providers/
    Microsoft.Compute/capacityReservationGroups/{crg}/capacityReservations/{cr}
  Body: { "sku": { "capacity": 0 } }
  Poll: GET CR → wait for provisioningState == Succeeded
  [VM continues running – no customer downtime at this step]

  — STEP 2: Clear Consumer VM's CRG reference (Path B – Consumer side) —
  PUT /subscriptions/{consumer_sub}/resourceGroups/{rg}/providers/
    Microsoft.Compute/virtualMachines/{vm_name}
  Body: { "properties": { "capacityReservationGroup": null } }
  Poll: GET VM → confirm capacityReservationGroup == null
  [VM continues running; CR SLA guarantee is now removed]
  △ Requires engine to hold customer-granted credentials / Managed Identity
    with VM contributor rights in the consumer subscription (see open item below)

  — STEP 3: Confirm disassociation —
  GET /capacityReservations/{cr} → confirm allocatedResourceCount decremented
  Log: VM_ImpactRecord { status: DISASSOCIATED, path: PathB,
    sla_impact: "CR SLA removed – VM continues running
(PathB)" }

  — PATH A FALLBACK (per-VM – if Path B Step 2 rejected by ARM) —
  POST ../virtualMachines/{vm_name}/deallocate → VM goes offline
  PUT ../virtualMachines/{vm_name} → capacityReservationGroup: null
  Log: VM_ImpactRecord { status: DISASSOCIATED, path: PathA,
    sla_impact: "VM deallocated – restart required by custom-
er" }
  Alert: PathAFallbackUsed (Severity: High) – operator notified; customer must re-
start VM manually

AFTER ALL VMs PROCESSED:

  — STEP 4: Expand DR CRG quantity —
  PUT /subscriptions/{dr_sub}/resourceGroups/{rg}/providers/
    Microsoft.Compute/capacityReservationGroups/{dr_crg}/capacityReservations/
{dr_cr}
  Body: { "sku": { "capacity": dr_crg_current_qty + freed_slots } }
  Poll: provisioningState == Succeeded

  — STEP 5: Update engine state —
  RegionalSnapshot:
    dr_crg_quantity      += freed_slots
    dr_crg_free_slots    += freed_slots
    dr_crg_coverage_ratio = dr_crg_quantity / potential_dr_demand (recalculated)
    nonprod_crg_quantity -= freed_slots
    nonprod_crg_effective_free recalculated
  QuotaGroup (NonProdDR):
    dr_quota_used_vcpu   += freed_slots × vCPU_per_instance
    nonprod_quota_used_vcpu -= freed_slots × vCPU_per_instance

  — STEP 6: Post-transfer audit alert —
  Alert: EmergencyCapacityTransferCompleted (Severity: Critical – DR audit record)
  Payload: slots_transferred, VMs_impacted, RTO_duration_minutes, correlation_id,
    path_a_fallbacks (count), quota_neutral: false

```

VMSS — Phase 1 Limitation

VMSS disassociation via Path B requires updating the **VMSS model** `capacityReservationGroup` property (a single ARM call affecting all instances simultaneously). This is structurally different from single-VM disassociation and carries a higher blast radius. For Phase 1:

- VMSS entries in `vm_disassociation_list` are **rejected by the engine** with a validation error.
- Operators must coordinate VMSS capacity release manually (customer notification required).
- A dedicated VMSS emergency disassociation path will be designed in a future iteration when design maturity allows full automation (see G-13 future scope).

Open Item — Consumer Credential Model

Tier 3 Step 2 (clearing the Consumer VM's CRG reference) requires the engine to call the Azure Compute API in **a customer-owned subscription**. This is a security architecture dependency:

Options to resolve before Tier 3 implementation:

1. **Managed Identity + RBAC delegation** — customer grants ACRME Managed Identity `Virtual Machine Contributor` rights during onboarding. Engine uses MI credentials at runtime. Preferred path.
2. **Cross-tenant service principal** — ACRME SP registered in customer tenant with delegated Compute rights. Higher onboarding friction; requires customer tenant consent.

This open item is a **Tier 3 implementation blocker** — Tier 3 cannot be built until the credential model is resolved and codified in the onboarding specification.

Integration with ACRME Engine

This design extends the existing placement engine (EPIC-07) with **region-selection semantics on top of the existing constraint evaluation**.

New API endpoint

POST /api/v1/placement/select-regions

Request:

```
{
  "customer_id":      "cust-xyz",
  "prod_region":      "eastus",
  "vm_sku":           "Standard_D4s_v3",
  "vm_count":         10,
  "policy_id":        "default-availability-first"
}
```

Response (202 → poll operation):

```
{
  "assignment": {
    "prod_region":      "eastus",
    "nonprod_region":   "australiaeast",
    "dr_region":        "eastus2"
  },
  "scores": {
    "nonprod_candidates": {"australiaeast": 0.765, "southeastasia": 0.523, "west-europe": 0.535},
    "dr_candidates":      {"westeurope": 0.571, "southeastasia": 0.559}
  },
  "policy_applied":     "default-availability-first",
  "operation_id":       "...",
  "snapshot_age_sec":   212
}
```

New/Extended **CustomerRegionAssignment** entity (Cosmos DB)

```
CustomerRegionAssignment {
  assignment_id:      GUID (PK)
  customer_id:        String
  prod_region:        String
  nonprod_region:     String
  dr_region:          String
  policy_id:          GUID (FK → PlacementPolicy)
  score_breakdown:    JSON ← α,β,γ,δ,ε per region per env; full audit trail
  snapshot_timestamp: DateTime
  created_at:         DateTime
  status:             Enum [Active, Superseded, Revoked]
}
```

Updated PlacementPolicy fields [UPDATED — Pass 2 and Pass 3]

Extend **PlacementPolicy.rules** (design Section 4.1) with:

```

{
  "weights": { "alpha": 0.30, "beta": 0.20, "gamma": 0.25, "delta": 0.15, "epsilon":
0.10 },
  "min_crg_score_threshold":      0.10,    // HC-2 floor
  // [UPDATED] Quota headroom floors are now group-type specific (replaces single
min_quota_headroom_vcpu)
  "min_prod_quota_headroom_vcpu": 20,      // HC-3: Prod Quota Group minimum headroom
before region is rejected
  "min_nonprod_quota_headroom_vcpu": 20,    // HC-3: NonProd effective headroom (floor-
adjusted) minimum
  "min_dr_headroom_vcpu":        16,      // HC-3: DR headroom (within NonProdDR
group) minimum
  "distribution_gamma_decay":     0.95,    // reduce  $\gamma$  weight for very large customer
counts (smoothing)
  "dr_buffer_pct":               0.50,    // 50% buffer (drives DR_Buffer_Score denom-
inator)
  "dr_ratio_min":                0.30,    // [NEW – Pass 2] lower bound of acceptable
DR coverage ratio (HC-6 floor)
  "dr_ratio_max":                0.40,    // [NEW – Pass 2] upper bound / target ceil-
ing; engine always optimises toward this
  // dr_ratio_target = [dr_ratio_min, dr_ratio_max] – range is [0.30, 0.40]; engine
targets dr_ratio_max
  "prod_quota_growth_buffer":     0.20,    // [NEW] Prod Quota Group size = CRG qty  $\times$ 
vCPU  $\times$  (1 + buffer)
  "nonprod_quota_growth_buffer": 0.20,    // [NEW] NonProdDR group NonProd portion
growth buffer
  "target_az_count":             3,
  "capacity_exhaustion_queue":   true,    // queue vs. hard fail on exhaustion

  // — STEADY STATE AUTO-INCREASE [NEW – Pass 3 / G-9]
  "dr_autoincrease_threshold":    0.35,    // [NEW] DR CRG trigger: fire when cover-
age_ratio < this
                                          // Must satisfy dr_ratio_min < threshold  $\leq$ 
dr_ratio_max
                                          // Default 0.35 (midpoint); engine acts be-
fore HC-6 fires at 0.30
  "prod_autoincrease_threshold":  0.20,    // [NEW] Prod CRG trigger: fire when
free_ratio < this (80% full)
  "nonprod_autoincrease_threshold": 0.20,  // [NEW] NonProd CRG trigger: fire when ef-
fective_free_ratio < this
  "autoincrease_cooldown_minutes": 30,     // [NEW] debounce: suppress repeat trigger
for same CRG within window
  "autoincrease_auto_approve":    false,   // [NEW] Phase A=false (approval required);
Phase B=true (self-managed)

  // — EMERGENCY CAPACITY TRANSFER [NEW – Pass 3 / G-10, G-12]
  "emergency_transfer_pct":       0.30,    // [NEW] fraction of potential_dr_demand
pre-staged as crisis headroom
                                          // emergency_transfer_headroom_vcpu = poten-
tial_dr_demand  $\times$  this  $\times$  vCPU
                                          // Default 0.30: DR CRG (40%) + headroom
(30%) = 70% combined crisis capacity

  "emergency_transfer_approval": {
    "tier1_auto_approve":        true,     // always automated –
no override
    "tier2_auto_approve":        false,    // Phase A: approval
required; Phase B: true
    "tier3_auto_approve":        false,    // always false – en-
forced in code, not config

```

```

    "tier2_approver_role": "ACRME.CapacityAdmin",
    "tier3_approver_role": "ACRME.EmergencyOperator", // elevated RBAC
    "tier3_dual_approval_required": true,
    "break_glass_role": "ACRME.SuperAdmin"           // single-approver
  override; always audit-logged
  }
}

```

Field deprecation note: `min_quota_headroom_vcpu` (single value) is **deprecated**. Engines running the new model must use the three env-type-specific fields above. For backward compatibility during migration, if only `min_quota_headroom_vcpu` is present, all three env-type floors default to its value.

Backlog Stories (new/extended) [UPDATED]

Quota Group Stories (EPIC-03 extension — must complete before E07-S12 is fully operational):

Story ID	Title	Priority	Size	Dependencies
E03-S09	Implement QuotaGroup entity in Cosmos DB; CRUD for group registration, member subscription management, derived field calculation	P0	M	E01-S02
E03-S10	Extend Quota Sync Worker: call Microsoft.Quota/groupQuotas API per group per region; decompose NonProdDR group into non-prod/dr components; recalculate dr_floor, effective_nonprod_ceiling, floor compliance	P0	L	E03-S01, E03-S09
E03-S11	Implement DR Floor protection enforcement: block NonProd CRG quantity increase when it would encroach on dr_floor_vcpu ; emit DRFloorViolationDetected alert	P0	M	E03-S10
E03-S12	Update quota pre-validation utility (E03-S03): use group headroom checks instead of per-sub-	P0	S	E03-S10, E03-S11

Story ID	Title	Priority	Size	Dependencies
	scription limits; enforce HC-7 (DR floor integrity)			
E03-S13	Update Quota_Score β sub-formula: group-level headroom per environment type; HC-3 revised + HC-7 added; update RegionalSnapshot Redis model with group fields	P1	M	E03-S10, E07-S12
E03-S14	Extend quota increase workflow (E03-S06): target Microsoft.Quota/groupQuotas endpoint; separate workflows for Prod group vs NonProdDR group	P1	M	E03-S06, E03-S09
E03-S15	Implement group-level chargeback split: NonProdDR group cost into NonProd and DR portions; attribute to customers proportionally	P2	M	E10-S04, E03-S09

Placement Engine Stories (EPIC-07):

Story ID	Title	Priority	Size	Dependencies
E07-S10	Implement <code>POST /place-ment/select-regions</code> endpoint; sequential Prod→NonProd→DR selection algorithm; score computation from regional snapshots	P1	L	E07-S01, E07-S07
E07-S11	Implement <code>CustomerRegionAssignment</code> Cosmos container; persist assignment + full score breakdown (including group quota fields) as audit record on every <code>select-regions</code> call	P1	M	E07-S10
E07-S12	Implement <code>RegionalSnapshot</code> Redis model with Quota Group fields (<code>prod_group_</code> , <code>non-prod_dr_group_</code> , <code>dr_floor_</code> , <code>non-prod_headroom_</code> , etc.); populate from reconciliation cycle; fallback to Cosmos on staleness. Depends on E03-S09, E03-S10 (group entity + sync worker) for group-	P1	M	E05-S01, E01-S04, E03-S09, E03-S10

Story ID	Title	Priority	Size	Dependencies
	level field population.			
E07-S13	Implement <code>Distribution_Score</code> and consumer-count tracking; increment <code>consumer_count_{env}</code> on assignment; decrement on customer offboarding	P1	M	E07-S11
E07-S14	Implement capacity exhaustion path: <code>RegionExhaustedError</code> , <code>pending_placement_queue</code> , re-evaluation on snapshot refresh, <code>CapacityExhaustedAlert</code> event	P1	M	E07-S10, E03-S06
E07-S15	Implement <code>RegionDistributionSaturated</code> alert: trigger when all candidate regions <code>Distribution_Score < 0.2</code>	P2	S	E07-S10
E07-S16	Extend <code>PlacementPolicy.rules</code> with new weight fields and thresholds including quota group fields (<code>min_prod_quota_headroom_vcpu</code> , <code>min_nonprod_quota_headroom_vcpu</code> ,	P1	S	E07-S07

Story ID	Title	Priority	Size	Dependencies
	<code>min_dr_headroom</code> <code>_vcpu ,</code> <code>dr_ratio_max ,</code> <code>prod/non-</code> <code>prod_quota_grow</code> <code>th_buffer); val-</code> <code>idate weights</code> <code>sum to 1.0 on</code> <code>CRUD; deprec-</code> <code>ate</code> <code>min_quota_headr</code> <code>oom_vcpu</code>			

Implementation Guidance

Phase 0 — Regional snapshot cache

Before implementing the algorithm, build the `RegionalSnapshot` Redis model (E07-S12). The reconciliation engine (E05-S01) must write updated snapshots on every cycle. This is the data feed the formula depends on.

Phase 1 — Score library (`shared/placement/`)

Implement each sub-score as a pure function: `crg_score(snap)` , `quota_score(snap)` , `distribution_score(snap, env_type)` , `dr_buffer_score(snap, env_type, policy)` , `zone_score(snap, policy)` . These are unit-testable with no Azure dependency. Cover edge cases: zero CRGs (returns 0.0), zero `quota_limit` (returns 0.0), all customers saturated (`Distribution_Score` approaches 0).

Phase 2 — Selection algorithm

Implement `select_regions()` consuming the score library. Validate HC-1 through HC-5 before scoring. Make the algorithm deterministic (same snapshot → same result) and log all intermediate values. The jitter must be seeded from `(customer_id, timestamp)` for reproducibility in replay.

Phase 3 — API and persistence

Wire `POST /placement/select-regions` through the saga framework (`OperationRecord` with before/after state). Update regional snapshot consumer counts atomically (Redis `HINCRBY` + Cosmos update).

Phase 4 — Policy tuning

Start with default weights. After 30-50 customer placements, analyse the score breakdown data in `OperationRecord`. Adjust γ (distribution weight) if regions are converging too slowly; increase δ (DR buffer weight) if DR buffer violations appear.

Tuning guidance for weight α (CRG capacity)

The `CRG_Score` is the most important signal when regions are unevenly provisioned. Set $\alpha \geq 0.30$ when capacity is tight. Reduce to 0.20 once regions reach steady-state utilization (40-60%) and distribution becomes the primary concern.

Tuning guidance for weight γ (distribution fairness)

γ is the self-correction mechanism. At $\alpha=0.30$, $\gamma=0.25$ gives distribution roughly equal importance to capacity. Increase γ toward 0.35 if score convergence results in uneven region loading. Decrease toward 0.15 if capacity constraints should dominate over fairness.

Decision Log

D1 — Sequential selection (Prod→NonProd→DR) vs. joint optimization

Decision: Sequential.

Alternatives considered: Joint optimization (select all three simultaneously using a combined objective function).

Trade-offs: Joint optimization is computationally more complex and requires solving a 3-assignment combinatorial problem (N choose 3 for 4 regions = 4 combinations — tractable, but adds complexity). Sequential is transparent, auditable, and deterministic. With only 3–4 regions the optimal joint solution and the sequential greedy result are identical in nearly all practical cases.

Impact: Simplicity wins. Revisit if region count exceeds 6.

D2 — Distribution_Score using absolute customer counts vs. relative percentages

Decision: Relative percentage (`customer_count / total_customers`).

Alternatives: Absolute count, normalized against region capacity.

Trade-offs: Relative percentage naturally scales as the customer base grows without recalibrating the weight bounds. Absolute counts would cause the `Distribution_Score` to drift toward 0 for all regions as T grows, reducing the weight's effectiveness.

Impact: Formula remains stable across growth phases.

D3 — CRG Weight uses mean vs. sum of CRGWeight per region

Decision: Mean (average).

Alternatives: Sum (total weight).

Trade-offs: Sum favors regions with more CRGs regardless of utilization quality. Mean rewards a region where each CRG is genuinely available, not just one with many but saturated CRGs.

Impact: More accurate signal of per-CRG availability quality.

D4 — Non-paired regions: geographic distance not encoded in formula

Decision: HC-4 (Separation Class) is a hard constraint, not a scored soft objective.

Rationale: With only 3–4 explicitly chosen non-paired regions, the operator has already made the geographic separation decision at infrastructure design time. The formula does not need to re-score geography — it assumes all configured regions are geographically independent by construction. If a future region is added that is geographically co-located with an existing region, HC-4 should reject it.

Impact: Simplifies the formula. Operators are responsible for region set composition.

D5 — 3-region edge case: only one valid choice for NonProd and DR

Decision: Still run the scoring algorithm; the “winner” is the only eligible candidate.

Rationale: Makes the code path uniform regardless of region count; the score output is still recorded

in the audit log and serves as a capacity health check (if the sole candidate has very low score, emit a capacity warning).

Impact: No code branching on region count; consistent auditability.

D6 — Two Quota Groups per region (Prod | NonProd+DR) vs. single shared quota pool vs. per-CRG quota tracking

Decision: Two quota groups per region — one Prod-only, one NonProd+DR shared.

Alternatives considered:

- Single shared quota pool: All three CRGs draw from one quota group. Simpler provisioning but breaks Prod isolation — a NonProd surge can consume quota that Prod CR creation needs. Rejected.
- Three separate quota groups (one per CRG): Maximum isolation but makes Tier 3 Emergency Capacity Transfer non-atomic — releasing NonProd quota goes to the NonProd group, but DR expansion draws from the DR group; a quota increase request to Azure may be needed in an emergency (hours of RTO impact). Rejected.
- Per-subscription quota tracking only (legacy): The original model. Retained as a legacy monitoring path but superseded for scoring and constraint checking. Cannot make Tier 3 quota-neutral without cross-subscription coordination risk.

Trade-offs: Two groups require an engine-enforced DR floor within the NonProdDR group (Azure does not natively support intra-group sub-reservations). This is a soft ceiling the engine must maintain continuously. The risk of DR floor violation due to a bug is real but mitigated by `DRFloorViolationDetected` alerting and operator gates on NonProd CRG scale operations.

Impact: Tier 3 Emergency Capacity Transfer becomes truly quota-neutral (ARM operations only, no Azure quota approval gate). Chargeback clarity improves — one cost record per group per region from Azure billing. `Quota_Score` must now be environment-type-aware (three formulas, one per env type).

D7 — DR floor uses `dr_ratio_max (0.40)` not current `dr_ratio`

Decision: DR floor is always sized at `potential_dr_demand × dr_ratio_max` using the **upper bound** of the DR ratio range (0.40), not the current operating ratio.

Rationale: If the floor is sized at the current ratio and the ratio is later increased (e.g. 0.30 → 0.40 due to a new SLA commitment), the floor is undersized and DR expansion could be blocked. Sizing at the upper bound ensures the floor never needs to grow due to ratio changes — only due to actual Prod demand growth.

Trade-off: Slightly more quota headroom is reserved for DR than strictly necessary at lower ratios. This is a deliberate over-reservation to protect DR reliability. The cost delta is small and reviewed quarterly.

Impact: `effective_nonprod_ceiling_vcpu` is slightly lower than the naive calculation would suggest. Captured in quota group sizing formulas and PlacementPolicy field `dr_ratio_max`.

D8 — HC-1 Constraint Removal: DR and NonProd may share a region

Decision: Remove the hard constraint `DR_region ≠ NonProd_region` from HC-1. NonProd and DR environments for a customer are permitted to share a region.

Alternatives considered:

- Keep `DR_region ≠ NonProd_region`: Maximum environment separation. Simple constraint — easier to reason about. Rejected because it prevents using NonProd CRG as overflow capacity for DR events, which is the design intent of the NonProdDR quota group co-location.

- Remove constraint, no compensating HC: Allows co-location but creates risk of NonProd CRG over-consumption leaving no DR overflow headroom. Rejected.
- Remove constraint, add HC-6 (selected): NonProd CRG's `effective_free` (after `dr_overflow_reserve`) is counted as available DR overflow headroom. HC-6 (`DR_COVERAGE_FLOOR`) enforces that this combined capacity is sufficient before DR placement is accepted.

Trade-offs: Co-location is now possible and scored. A region hosting both NonProd and DR for the same customer must satisfy HC-6 and produce a high enough `PS_DR` score to win selection. The 3-region pool constraint (Prod always isolated) is preserved — only the NonProd/DR pairing constraint is removed.

Impact: HC-1 updated (line 228); R2 updated (line 51); HC-6 added; `candidates_dr` in the selection algorithm now excludes only Prod, not NonProd. `RegionalSnapshot` per-CRG-type fields are required to populate `nonprod_crg_effective_free` for HC-6 evaluation.

D9 — Env-Type-Specific Placement Score Formulas (`PS_Prod` / `PS_NonProd` / `PS_DR`)

Decision: Replace the single generic `PS(r, E)` formula with three environment-type-specific formulas. `PS_Prod` is added as a new formula (Prod scoring did not previously exist). `PS_NonProd` and `PS_DR` replace the former `PS(r, NonProd)` and `PS(r, DR)`.

Alternatives considered:

- Single generic formula with `env_type` flag: Simpler implementation — one function, one weight table. Rejected because α (CRG capacity signal) and δ (DR readiness / overflow health) have fundamentally different meanings per env type. A single formula either uses wrong semantics or requires so many conditional branches it effectively becomes three formulas anyway.
- Two formulas (NonProd + DR) plus `PS_Prod` as validation only: Keeps selection algorithm unchanged; `PS_Prod` used only post-selection. Rejected — the user confirmed `PS_Prod` must be usable for both recommendation and validation, requiring a consistent scoring path.
- Three formulas with shared weights (selected): Same default weight values ($\alpha=0.30$, $\beta=0.20$, $\gamma=0.25$, $\delta=0.15$, $\epsilon=0.10$) but α and δ have different semantic definitions per env type. Maximises formula consistency while enabling per-env-type optimisation.

Trade-offs: Requires per-CRG-type fields in `RegionalSnapshot` (new D9 dependency on E03-S10 extension). `compute_placement_score` pseudocode dispatches by `env_type`. Worked examples must be updated to use per-CRG-type inputs (backlog G-7, not yet complete). `CRG_Score` (RCW) is demoted from primary scoring input to monitoring-only signal.

Impact: Placement Scoring Formula section fully restructured. Three new pseudocode functions added (`compute_ps_prod`, `compute_ps_nonprod`, `compute_ps_dr`). `RegionalSnapshot` extended with 16 per-CRG-type fields. `PlacementPolicy.rules` extended with `dr_ratio_min`. Backlog stories E07-S16 and E03-S10 inherit scope extension.

D10 — Two Separate Operating Systems: Steady State vs Emergency Capacity Transfer

Decision: Capacity lifecycle management is split into two architecturally separate operating systems sharing the same CR/CRG infrastructure. **Steady State Capacity Lifecycle Management** handles

organic growth via the reconciliation loop and `CapacityIncreaseRequest` workflow. **Emergency Capacity Transfer** handles crisis-only operations when a DR event is declared and primary region is down.

Alternatives considered:

- Unified capacity management system: Single engine handles both steady-state growth and crisis operations with mode flags. Rejected — mode flags create complex conditional logic, risk triggering crisis operations (VM disassociation) during routine reconciliation, and obscure the operational boundary between routine and destructive actions.
- Emergency transfer as an extension of steady-state auto-increase: Auto-increase escalates to emergency tiers when headroom is exhausted. Rejected — this conflates different approval models (policy-driven growth vs. operator-gated crisis response) and could allow automated emergency operations without explicit DR event declaration.
- Two separate systems with explicit mode gate (selected): `engine_mode` field (`STEADY_STATE` vs `DR_EVENT_ACTIVE`) enforces operational boundary in code. Auto-increase trigger is suppressed during DR events; Emergency Transfer is rejected outside DR events. Clear audit trail — all operations tagged with operating mode.

Trade-offs: Requires `engine_mode` to be a formal engine state stored in Cosmos DB and propagated to the reconciliation loop on each cycle. Mode transitions (declaring/resolving a DR event) must be operator-gated — not automatic.

Impact: Two new top-level sections added to the design document. `CapacityIncreaseRequest` is a new named entity (Cosmos DB). `EmergencyCapacityTransfer` is a new named API operation. `PlacementPolicy.rules` extended with auto-increase thresholds, cooldown, `emergency_transfer_pct`, and `emergency_transfer_approval` block (8 new fields).

D11 — Emergency Transfer Tier Model (Tier 1 / Tier 2 / Tier 3) and Relabelling

Decision: Define a three-tier escalation model for emergency crisis operations. Tier 1 (DirectExpansion) is fully automated. Tier 2 (QuotaNeutralTransfer) is policy-gated and replaces what was formerly labelled “Tier 3 Emergency Transfer” in earlier document versions. Tier 3 (DestructiveTransfer) involves operator-provided VM disassociation and requires dual approval plus elevated RBAC.

Relabelling: The prior document’s “Tier 3 Emergency Transfer” referred to the quota-neutral Non-Prod→DR CR quantity transfer (no VM disruption beyond SLA removal). Under this model that operation is correctly **Tier 2**, because it does not touch VM execution state. “Tier 3” is now reserved exclusively for operations that affect VM-to-CRG associations via Path B (or Path A fallback) disassociation.

Alternatives considered:

- Two tiers only (automated + destructive): Simpler but loses the intermediate Tier 2 path, which is quota-neutral and far lower risk than VM disassociation. Rejected — the quota-neutral path is a critical intermediate escalation step that avoids Tier 3 in the majority of crisis scenarios.
- Four tiers (add VMSS-specific tier): VMSS disassociation is structurally different from single-VM Path B and carries higher blast radius. Deferred — VMSS emergency path is a Phase 1 limitation, not a separate tier. Will be re-evaluated when VMSS disassociation is designed.
- Three tiers with clear boundary between quota-impacting and VM-impacting (selected): Tier 1 and Tier 2 are quota operations only (no VM state change). Tier 3 is the only tier that modifies VM-to-CRG associations. VM disassociation is always operator-supplied (user provides `vm_disassociation_list`) — no automated VM selection in Phase 1.

Trade-offs: Tier 3 is a Tier 3 implementation blocker on the consumer credential model (Managed Identity vs. cross-tenant SP). VMSS entries in `vm_disassociation_list` are rejected by the engine in Phase 1. Break-glass role (`ACRME.SuperAdmin`) allows single-approver override — must be audited and rare.

Impact: Emergency Capacity Transfer section added. `EmergencyCapacityTransferRequest` / `EmergencyCapacityTransferResponse` schemas defined. `VM_DisassociationTarget` , `ExecutionStep` , `VM_ImpactRecord` sub-schemas defined. Tier 3 execution sequence (6-step Path B default with Path A fallback per VM) documented. VMSS Phase 1 limitation and consumer credential open item formally recorded.